Formatting Ranges

Document #:	P2286R4
Date:	2021-12-17
Project:	Programming Language C++
Audience:	LEWG

Reply-to: Barry Revzin <barry.revzin@gmail.com>

Contents

§ 1 Revision History

Since [P2286R3], several major changes:

— Removed the special `pair`/`tuple` parsing for individual elements. This proved complicated and illegible, and led to having to deal with more issues that would make this paper harder to make it for C++23.

— Adding sections on dynamic and static delimiters for ranges. Removing `std::format_join` in their favor.

— Renaming `format_as_debug` to `set_debug_format` (since it's not actually *formatting* anything, it's just setting up)

— Discussing `std::filesystem::path`

Since [P2286R2], several major changes:

— This paper assumes the adoption of [P2418R0], which affects how non-`const`-iterable views are handled. This paper now introduces two concepts (`formattable` and `const_formattable`) instead of just one.

— Extended discussion and functionality for various representations, including how to quote strings properly and how to format associative ranges.

— Introduction of format specifiers of all kinds and discussion of how to make them work more broadly.

— Removed the wording, since the priority is the design.

Since [P2286R1], adding a sketch of wording.

[P2286R0] suggested making all the formatting implementation-defined. Several people reached out to me suggesting in no uncertain terms that this is unacceptable. This revision lays out options for such formatting.

§ 2 Introduction

[LWG3478] addresses the issue of what happens when you split a string and the last character in the string is the delimiter that you are splitting on. One of the things I wanted to look at in research in that issue is: what do *other* languages do here?

For most languages, this is a pretty easy proposition. Do the split, print the results. This is usually only a few lines of code.

Python:

```
print("xyx".split("x"))
```

outputs

```
['', 'y', '']
```

Java (where the obvious thing prints something useless, but there's a non-obvious thing that is useful):

```
import java.util.Arrays;

class Main {
  public static void main(String args[]) {
    System.out.println("xyx".split("x"));
    System.out.println(Arrays.toString("xyx".split("x")));
  }
}
```

outputs

```
[Ljava.lang.String;@76ed5528
[, y]
```

Rust (a couple options, including also another false friend):

```
use itertools::Itertools;

fn main() {
    println!("{:?}", "xyx".split('x'));
    println!("[{}]", "xyx".split('x').format(", "));
    println!("{:?}", "xyx".split('x').collect::<Vec<_>>());
}
```

outputs

```
Split(SplitInternal { start: 0, end: 3, matcher: CharSearcher { haystack: "xyx",
        finger: 0, finger_back: 3, needle: 'x', utf8_size: 1, utf8_encoded: [120,
        0, 0, 0] }, allow_trailing_empty: true, finished: false })
[, y, ]
["", "y", ""]
```

Kotlin:

```
fun main() {
    println("xyx".split("x"));
}
```

outputs

```
[, y, ]
```

Go:

```
package main
import "fmt"
import "strings"

func main() {
    fmt.Println(strings.Split("xyx", "x"));
}
```

outputs

```
[ y ]
```

JavaScript:

```
console.log('xyx'.split('x'))
```

outputs

```
[ '', 'y', '' ]
```

And so on and so forth. What we see across these languages is that printing the result of split is pretty easy. In most cases, whatever the print mechanism is just works and does something meaningful. In other cases, printing gave me something other than what I wanted but some other easy, provided mechanism for doing so.

Now let's consider C++.

```
#include <iostream>
#include <string>
#include <ranges>
#include <format>

int main() {
    // need to predeclare this because we can't split an rvalue string
    std::string s = "xyx";
    auto parts = s | std::views::split('x');

    // nope
    std::cout << parts;

    // nope (assuming std::print from P2093)
    std::print("{}", parts);


    std::cout << "[";
    char const* delim = "";
    for (auto part : parts) {
        std::cout << delim;

        // still nope
        std::cout << part;

        // also nope
        std::print("{}", part);

        // this finally works
        std::ranges::copy(part, std::ostream_iterator<char>(std::cout));

        // as does this
        for (char c : part) {
            std::cout << c;
        }
        delim = ", ";
    }
    std::cout << "]\n";
}
```

This took me more time to write than any of the solutions in any of the other languages. Including the Go solution, which contains 100% of all the lines of Go I've written in my life.

Printing is a fairly fundamental and universal mechanism to see what's going on in your program. In the context of ranges, it's probably the most useful way to see and understand what the various range adapters actually do. But none of these things provides an `operator<<` (for `std::cout`) or a formatter specialization (for `format`). And the further problem is that as a user, I can't even do anything about this. I can't just provide an `operator<<` in

`namespace std` or a very broad specialization of `formatter` - none of these are program-defined types, so it's just asking for clashes once you start dealing with bigger programs.

The only mechanisms I have at my disposal to print something like this is either

1. nested loops with hand-written delimiter handling (which are tedious and a bad solution), or
2. at least replace the inner-most loop with a `ranges::copy` into an output iterator (which is more differently bad), or
3. Write my own formatting library that I *am* allowed to specialize (which is not only bad but also ridiculous)
4. Use `fmt::format`.

§ 2.1 Implementation Experience

That's right, there's a fourth option for C++ that I haven't shown yet, and that's this:

```
#include <ranges>
#include <string>
#include <fmt/ranges.h>

int main() {
    std::string s = "xyx";
    auto parts = s | std::views::split('x');

    fmt::print("{}\n", parts);
    fmt::print("<<{}>>\n", fmt::join(parts, "--"));
}
```

outputting

```
[[], ['y'], []]
<<[]--['y']--[]>>
```

And this is great! It's a single, easy line of code to just print arbitrary ranges (include ranges of ranges).

And, if I want to do something more involved, there's also `fmt::join`, which lets me specify both a format specifier and a delimiter. For instance:

```
std::vector<uint8_t> mac = {0xaa, 0xbb, 0xcc, 0xdd, 0xee, 0xff};
fmt::print("{:02x}\n", fmt::join(mac, ":"));
```

outputs

```
aa:bb:cc:dd:ee:ff
```

`fmt::format` (and `fmt::print`) solves my problem completely. `std::format` does not, and it should.

§ 3 Proposal Considerations

The Ranges Plan for C++23 [P2214R1] listed as one of its top priorities for C++23 as the ability to format all views. Let's go through the issues we need to address in order to get this functionality.

§ 3.1 What types to print?

The standard library is the only library that can provide formatting support for standard library types and other broad classes of types like ranges. In addition to ranges (both the conrete containers like `vector<T>` and the range adaptors like `views::split`), there are several very commonly used types that are currently not printable.

The most common and important such types are `pair` and `tuple` (which ties back into Ranges even more closely once we adopt `views::zip` and `views::enumerate`). `fmt` currently supports printing such types as well:

```
fmt::print("{}\n", std::pair(1, 2));
```

outputs

```
(1, 2)
```

Another common and important set of types are `std::optional<T>` and `std::variant<Ts...>`. `fmt` does not support printing any of the sum types. There is not an obvious representation for them in C++ as there might be in other languages (e.g. in Rust, an `Option<i32>` prints as either `Some(42)` or `None`, which is also the same syntax used to construct them).

However, the point here isn't necessarily to produce the best possible representation (users who have very specific formatting needs will need to write custom code anyway), but rather to provide something useful. And it'd be useful to print these types as well. However, given that `optional` and `variant` are both less closely related to Ranges than `pair` and `tuple` and also have less obvious representation, they are less important.

§ 3.2 Detecting whether a type is formattable

We need to be able to conditionally provide formatters for generic types. `vector<T>` needs to be formattable when `T` is formattable. `pair<T, U>` needs to be formattable when `T` and `U` are formattable. In order to do this, we need to provide a proper `concept` version of the formatter requirements that we already have.

This paper suggests the following:

```
template<class T, class charT>
concept formattable-impl =
    semiregular<formatter<T, charT>> &&
    requires (formatter<T, charT> f,
              const formatter<T, charT> cf,
              T t,
              basic_format_context<fmt-iter-for<charT>, charT> fc,
              basic_format_parse_context<charT> pc) {
        { f.parse(pc) } -> same_as<basic_format_parse_context<charT>::iterator>;
        { cf.format(t, fc) } -> same_as<fmt-iter-for<charT>>;
    };

template<class T, class charT>
concept formattable = formattable-impl<remove_cvref_t<T>, charT>;
```

The broad shape of this concept is just taking the Formatter requirements and turning them into code. There are a few important things to note though:

— We don't specify what the iterator type is of `format_context` or `wformat_context`, the expectation is that formatters accept any iterator. As such, it is unspecified in the concept *which* iterator will be checked - simply that it is *some* `output_iterator<charT const&>`. Implementations could use `format_context::iterator` and `wformat_context::iterator`, or they could have a bespoke minimal iterator dedicated for concept checking.

— `cf.format(t, fc)` is called on a `const formatter` (see [LWG3636])

— `cf.format(t, fc)` is called specifically on `T`, not a `const T`. Even if the typical formatter specialization will take its object as `const T&`. This is to handle cases like ranges that are not `const`-iterable.

— `formattable<T const, char>` could be `true` even if you can't actually format a `T const`. I'm not sure that this will be a significant issue in practice.

§ 3.3 What representation?

There are several questions to ask about what the representation should be for printing. I'll go through each kind in turn.

§ 3.3.1 `vector` (and other ranges)

Should `std::vector<int>{1, 2, 3}` be printed as `{1, 2, 3}` or `[1, 2, 3]`? At the time of [P2286R1], `fmt` used `{}`s but changed to use `[]`s for consistency with Python (400b953f).

Even though in C++ we initialize `vectors` (and, generally, other containers as well) with `{}`s while Python's uses `[1, 2, 3]` (and likewise Rust has `vec![1, 2, 3]`), `[]` is typical representationally so seems like the clear best choice here.

§ 3.3.2 `pair` and `tuple`

Should `std::pair<int, int>{4, 5}` be printed as `{4, 5}` or `(4, 5)`? Here, either syntax can claim to be the syntax used to initialize the `pair`/`tuple`. `fmt` has always printed these types with `()`s, and this is also how Python and Rust print such types. As with using `[]` for ranges, `()` seems like the common representation for tuples and so seems like the clear best choice.

§ 3.3.3 `map` and `set` (and other associative containers)

Should `std::map<int, int>{{1, 2}, {3, 4}}` be printed as `[(1, 2), (3, 4)]` (as follows directly from the two previous choices) or as `{1: 2, 3: 4}` (which makes the *association* clearer in the printing)? Both Python and Rust print their associating containers this latter way.

The same question holds for sets as well as maps, it's just a question for whether `std::set<int>{1, 2, 3}` prints as `[1, 2, 3]` (i.e. as any other range of `int`) or `{1, 2, 3}`?

If we print `maps` as any other range of pairs, there's nothing left to do. If we print `maps` as associations, then we additionally have to answer the question of how user-defined associative containers can get printed in the same way. Hold onto this thought for a minute.

§ 3.3.4 `char` and `string` (and other string-like types) in ranges or tuples

Should `pair<char, string>('x', "hello")` print as `(x, hello)` or `('x', "hello")`? Should `pair<char, string>('y', "with\n\"quotes\"")` print as:

```
(y, with
"quotes")
```

or

```
('y', "with\n\"quotes\"")
```

While `char` and `string` are typically printed unquoted, it is quite common to print them quoted when contained in tuples and ranges. This makes it obvious what the actual elements of the range and tuple are even when the string/char contains characters like comma or space. Python, Rust, and `fmt` all do this. Rust escapes internal strings, so prints as `('y', "with\n\"quotes\"")` (the Rust implementation of `Debug` for `str` can be found here which is implemented in terms of `escape_debug_ext`). Following discussion of this paper and this design, Victor Zverovich implemented in this `fmt` as well.

Escaping is the most desirable default behavior, and the specific escaping behavior is described here.

Also, `std::string` isn't the only string-like type: if we decide to print strings quoted, how do users opt in to this behavior for their own string-like types? And `char` and `string` aren't the only types that may desire to have some kind of *debug* format and some kind of regular format, how to differentiate those?

Moreover, it's all well and good to have the default formatting option for a range or tuple of strings to be printing those strings escaped. But what if users want to print a range of strings *unescaped*? I'll get back to this.

3.3.5 `filesystem::path`

We have a paper, [P1636R2], that proposes `formatter` specializations for a different subset of library types: `basic_streambuf`, `bitset`, `complex`, `error_code`, `filesystem::path`, `shared_ptr`, `sub_match`, `thread::id`, and `unique_ptr`. Most of those are neither ranges nor tuples, so that paper doesn't overlap with this one.

Except for one: `filesystem::path`.

During the SG16 discussion of P1636, they took a poll that:

> Poll 1: Recommend removing the filesystem::path formatter from P1636 "Formatters for library types", and specifically disabling filesystem::path formatting in P2286 "Formatting ranges", pending a proposal with specific design for how to format paths properly.

SF	F	A	N	SA
5	5	1	0	0

`filesystem::path` is kind of an interesting range, since it's a range of `path`. As such, checking to see if it would be formattable as this paper currently does would lead to constraint recursion:

```
template <range R>
    requires formattable<range_reference_t<R>>
struct formatter<R>
    : range_formatter<range_reference_t<R>>
{ };
```

For `R=filesystem::path`, `range_reference_t<R>` is also `filesystem::path`. Which means that our constraint for `formatter<fs::path>` requires `formattable<fs::path>` Looking at the suggested concept, the first check we will do is to verify that `formatter<fs::path>` is `semiregular`. But we're currently in the process of instantiating `formatter<fs::path>`, it is still incomplete. Hard error.

In order to handle this case properly, we could do what SG16 suggested:

```
template <>
struct formatter<filesystem::path>;
```

But this only handles `std::filesystem::path` and would not handle other ranges-of-self (the obvious example here is `boost::filesystem::path`). So instead, this paper proposes that we first reject ranges-of-self:

```
template <range R>
    requires (not same_as<remove_cvref_t<range_reference_t<R>>, R>)
         and formattable<range_reference_t<R>>
struct formatter<R>
    : range_formatter<range_reference_t<R>>
{ };
```

3.3.6 Format Specifiers

One of (but hardly the only) the great selling points of `format` over iostreams is the ability to use specifiers. For instance, from the `fmt` documentation:

```
fmt::format("{:<30}", "left aligned");
// Result: "left aligned                  "
fmt::format("{:>30}", "right aligned");
// Result: "                 right aligned"
fmt::format("{:^30}", "centered");
// Result: "           centered           "
fmt::format("{:*^30}", "centered");  // use '*' as a fill char
// Result: "***********centered***********"
```

Earlier revisions of this paper suggested that formatting ranges and tuples would accept no format specifiers, but there indeed are quite a few things we may want to do here (as by Tomasz Kamiński and Peter Dimov):

— Formatting a range of pairs as a map (the `key: value` syntax rather than the `(key, value)` one)

— Formatting a range of chars as a string (i.e. to print `hello` or `"hello"` rather than `['h', 'e', 'l', 'l', 'o']`)

But these are just providing a specifier for how we format the range itself. How about how we format the elements of the range? Can I conveniently format a range of integers, printing their values as hex? Or as characters? Or print a range of chrono time points in whatever format I want? That's fairly powerful.

The problem is how do we actually *do that*. After a lengthy discussion with Peter Dimov, Tim Song, and Victor Zverovich, this is what we came up with. I'll start with a table of examples and follow up with a more detailed explanation.

Instead of writing a bunch of examples like `print("{:?}\n", v)`, I'm just displaying the format string in one column (the `"{:?}"` here) and the argument in another (the `v`):

Format String	Contents	Formatted Output
{:}	42	42
{:#x}	42	0x2a
{}	"h\tllo"s	h llo
{:?}	"h\tllo"s	"h\tllo"
{}	vector{"h\tllo"s, "world"s}	["h\tllo", "world"]
{:}	vector{"h\tllo"s, "world"s}	["h\tllo", "world"]
{::}	vector{"h\tllo"s, "world"s}	[h llo, world]
{:*^14}	vector{"he"s, "wo"s}	*["he", "wo"]*

Format String	Contents	Formatted Output
{::*^14}	vector{"he"s, "wo"s}	[******he******, ******wo******]
{}	vector<char>{'H', '\t', 'l', 'l', 'o'}	['H', '\t', 'l', 'l', 'o']
{::}	vector<char>{'H', '\t', 'l', 'l', 'o'}	[H, , l, l, o]
{::c}	vector<char>{'H', '\t', 'l', 'l', 'o'}	[H, , l, l, o]
{::?c}	vector<char>{'H', '\t', 'l', 'l', 'o'}	['H', '\t', 'l', 'l', 'o']
{::d}	vector<char>{'H', '\t', 'l', 'l', 'o'}	[72, 9, 108, 108, 111]
{::#x}	vector<char>{'H', '\t', 'l', 'l', 'o'}	[0x48, 0x9, 0x6c, 0x6c, 0x6f]
{:s}	vector<char>{'H', '\t', 'l', 'l', 'o'}	H llo
{:?s}	vector<char>{'H', '\t', 'l', 'l', 'o'}	"H\tllo"
{}	pair{42, "h\tllo"s}	(42, "h\tllo")
{}	vector{pair{42, "h\tllo"s}}	[(42, "h\tllo")]
{:m}	vector{pair{42, "h\tllo"s}}	{42: "h\tllo"}
{:m:}	vector{pair{42, "h\tllo"s}}	{42: h llo}
{}	vector{vector{'a'}, vector{'b', 'c'}}	[['a'], ['b', 'c']]
{::?s}	vector{vector{'a'}, vector{'b', 'c'}}	["a", "bc"]
{:::d}	vector{vector{'a'}, vector{'b', 'c'}}	[[97], [98, 99]]

3.3.7 Explanation of Added Specifiers

3.3.7.1 The debug specifier ?

`char` and `string` and `string_view` will start to support the `?` specifier. This will cause the character/string to be printed as quoted (characters with `'` and strings with `"`) and all characters to be escaped, as described earlier.

This facility will be generated by the formatters for these types providing an addition member function (on top of `parse` and `format`):

```
void set_debug_format();
```

Which other formatting types may conditionally invoke when they parse a `?`. For instance, since the intent is that range formatters print escaped by default, the logic for a simple range formatter that accepts no specifiers might look like this (note that this paper is proposing something more complicated than this, this is just an example):

```
template <typename V>
struct range_formatter {
```

```
    std::formatter<V> underlying;

    template <typename ParseContext>
    constexpr auto parse(ParseContext& ctx) {
        // ensure that the format specifier is empty
        if (ctx.begin() != ctx.end() && *ctx.begin() != '}') {
            throw std::format_error("invalid format");
        }

        // ensure that the underlying type can parse an empty specifier
        auto out = underlying.parse(ctx);

        // conditionally format as debug, if the type supports it
        if constexpr (requires { underlying.set_debug_format(); }) {
            underlying.set_debug_format();
        }
        return out;
    }

    template <typename R, typename FormatContext>
        requires
        std::same_as<std::remove_cvref_t<std::ranges::range_reference_t<R>>, V>
    constexpr auto format(R&& r, FormatContext& ctx) {
        auto out = ctx.out();
        *out++ = '[';
        auto first = std::ranges::begin(r);
        auto last = std::ranges::end(r);
        if (first != last) {
            // have to format every element via the underlying formatter
            ctx.advance_to(std::move(out));
            out = underlying.format(*first, ctx);
            for (++first; first != last; ++first) {
                *out++ = ',';
                *out++ = ' ';
                ctx.advance_to(std::move(out));
                out = underlying.format(*first, ctx);
            }
        }
        *out++ = ']';
        return out;
    }
};
```

3.3.7.2 Range specifiers

Range format specifiers come in two kinds: specifiers for the range itself and specifiers for the underlying elements of the range. They must be provided in order: the range specifiers (optionally), then if desired, a colon and then the underlying specifier (optionally). For instance:

specifier	meaning
`{}`	No specifiers
`{:}`	No specifiers
`{:<10}`	The whole range formatting is left-aligned, with a width of 10
`{:*^20}`	The whole range formatting is center-aligned, with a width of 20, padded with `*`s
`{:m}`	Apply the `m` specifier to the range (which must be a range of pair or 2-tuple)

specifier	meaning
`{::d}`	Apply the `d` specifier to each element of the range
`{:?s}`	Apply the `?s` specifier to the range (which must be a range of char)

There are only a few top-level range-specific specifiers proposed:

— `s`: for ranges of char, only: formats the range as a string.

— `?s` for ranges of char, only: same as `s` except will additionally quote and escape the string

— `m`: for ranges of `pairs` (or `tuples` of size 2) will format as `{k1: v1, k2: v2}` instead of `[(k1, v1), (k2, v2)]` (i.e. as a `map`).

— `e`: will format without the brackets. This will let you, for instance, format a range as `a, b, c` or `{a, b, c}` or `(a, b, c)` or however else you want, simply by providing the desired format string. If printing a normal range, the brackets removed are `[]`. If printing as a map, the brackets removed are `{}`. If printing as a quoted string, the brackets removed are the `""`s (but escaping will still happen).

— `d`: either a dynamic delimiter or static delimiter, depending on what follows the `d`. See those sections for more detail.

Additionally, ranges will support the same fill/align/width specifiers as in *std-format-spec*, for convenience and consistency.

If no element-specific formatter is provided (i.e. there is no inner colon - an empty element-specific formatter is still an element-specific formatter), the range will be formatted as debug. Otherwise, the element-specific formatter will be parsed and used.

To revisit a few rows from the earlier table:

Format String	Contents	Formatted Output
`{}`	`vector<char>{'H', '\t', 'l', 'l', 'o'}`	`['H', '\t', 'l', 'l', 'o']`
`{::}`	`vector<char>{'H', '\t', 'l', 'l', 'o'}`	`[H,  , l, l, o]`
`{::?c}`	`vector<char>{'H', '\t', 'l', 'l', 'o'}`	`['H', '\t', 'l', 'l', 'o']`
`{::d}`	`vector<char>{'H', '\t', 'l', 'l', 'o'}`	`[72, 9, 108, 108, 111]`
`{::#x}`	`vector<char>{'H', '\t', 'l', 'l', 'o'}`	`[0x48, 0x9, 0x6c, 0x6c, 0x6f]`
`{:s}`	`vector<char>{'H', '\t', 'l', 'l', 'o'}`	`H  llo`
`{:?s}`	`vector<char>{'H', '\t', 'l', 'l', 'o'}`	`"H\tllo"`
`{}`	`vector{vector{'a'}, vector{'b', 'c'}}`	`[['a'], ['b', 'c']]`
`{::?s}`	`vector{vector{'a'}, vector{'b', 'c'}}`	`["a", "bc"]`

Format String	Contents	Formatted Output
`{:::d}`	`vector{vector{'a'}, vector{'b', 'c'}}`	`[[97], [98, 99]]`

The second row is not printed quoted, because an empty element specifier is provided. We assume that if the user explicitly provides a format specifier (even if it's empty), that they want control over what they're doing. The third row is printed quoted again because it was explicitly asked for using the `?c` specifier, applied to each character.

The last row, `:::d`, is parsed as:

	top level outer vector		top level inner vector		inner vector each element
`:`	(none)	`:`	(none)	`:`	`d`

That is, the `d` format specifier is applied to each underlying `char`, which causes them to be printed as integers instead of characters.

Note that you can provide both a fill/align/width specifier to the range itself as well as to each element:

Format String	Contents	Formatted Output
`{}`	`vector<int>{1, 2, 3}`	`[1, 2, 3]`
`{::*^5}`	`vector<int>{1, 2, 3}`	`[**1**, **2**, **3**]`
`{:o^17}`	`vector<int>{1, 2, 3}`	`oooo[1, 2, 3]oooo`
`{:o^29:*^5}`	`vector<int>{1, 2, 3}`	`oooo[**1**, **2**, **3**]oooo`

3.3.7.3 Dynamic Delimiter for Ranges

Let's say I have a `vector<uint8_t>` that I wish to format as a MAC address. That is, I want to print every element with `"02x"`, delimited with `":"` (rather than the default `", "`), and without the surrounding square brackets.

I showed an example of how to do this earlier using `{fmt}`:

```
std::vector<uint8_t> mac = {0xaa, 0xbb, 0xcc, 0xdd, 0xee, 0xff};
fmt::print("{}\n", mac);                      // [170, 187, 204, 221, 238, 255]
fmt::print("{:02x}\n", fmt::join(mac, ":")); // aa:bb:cc:dd:ee:ff
```

However, if we're going to add more support for adding specifiers to ranges, that suggests a potential alternate avenue. We could add a dynamic delimiter in the same way that we support dynamic width in other contexts. That is:

```
fmt::print("{:ed{}:02x}", mac, ":"); // aa:bb:cc:dd:ee:ff
```

Here, `":ed{}"` are the specifiers for the top-level vector, and then `":02x"` are the specifiers for the underlying element. The `e` specifier (for empty brackets) avoids printing the `[]`s and then the `d` specifier (for delimiter) is followed by which argument to get the delimiter out of (`{}` for auto-numbering, could also have been `{1}` in this example).

Perhaps `e` is implicit with `d`, perhaps not.

The question is, there are ultimately two ways that we could format this mac address as a result of this paper:

```
fmt::print("{:02x}\n", fmt::join(mac, ":")); // aa:bb:cc:dd:ee:ff
fmt::print("{:ed{}:02x}\n", mac, ":");       // aa:bb:cc:dd:ee:ff
```

Do we want to pursue:

1. Just `fmt::join`?
2. Just dynamic delimiter?
3. Both?

The dynamic delimiter approach is more cryptic. The `join` approach arguably has the advantage of making it more clear what the delimiter is and how it's used, whereas in the dynamic delimiter approach it's just… wherever. I'll discuss static delimiters shortly.

The dynamic delimiter approach is also limited to *just* allowing `charT`, `charT const*`, and `basic_string_view<charT>` (and maybe `basic_string<charT>`) as delimiter types. The `fmt::join` approach would allow any type convertible to `basic_string_view<charT>`. This is a consequence of `fmt::join` being a function that accepts a `string_view` argument, while going through `format` directly can't do any sort of conversions - we have to use the type-erased arguments, and we simply cannot know if some user-defined type would have been convertible to `string_view`.

But the dynamic delimiter approach has advantages too.

First, it naturally nests. So if I wanted to format a *range* of mac addresses, I can do that:

```
// one mac
fmt::print("{:ed{}:02x}\n", one_mac, ":");
// range of macs
fmt::print("{::ed{}:02x}\n", some_macs, ":");
// range of range of macs
fmt::print("{:::ed{}:02x}\n", uber_macs, ":");
// range of range of macs, providing all three delimiters
fmt::print("{:ed{}:ed{}:02x}\n", uber_macs, "++", "**", ":");
```

Whereas this is much more awkward with `fmt::join`:

```
// one mac
fmt::print("{:02x}\n", fmt::join(one_mac, ":"));
// range of macs
fmt::print("{::02x}\n",
    some_macs | std::views::transform([](auto&& m){
        return fmt::join(m, ":");
    }));
// range of range of macs
fmt::print("{:::02x}\n",
    uber_macs | std::views::transform([](auto&& m){
        return m | std::views::transform([](auto&& m2){
            return fmt::join(m2, ":");
        });
    }));
// range of range of macs, providing all three delimiters
fmt::print("{:02x}\n",
    fmt::join(uber_macs | std::views::transform([](auto&& m){
        return fmt::join(m | std::views::transform([](auto&& m2){
```

```
            return fmt::join(m2, ":");
        }), "**");
    }),
    "++"));
```

The dynamic delimiter approach also supports more functionality. If I want to center-align the mac address and pad it with asterisks like I've been doing with every other example (for instance), that's just more specifiers as compared with another call to `format`:

```
fmt::print("{:*^23ed{}:02x}\n", mac, ":");                          //
        ***aa:bb:cc:dd:ee:ff***
fmt::print("{:*^23}\n", fmt::format("{:02x}", fmt::join(mac, ":"))); //
        ***aa:bb:cc:dd:ee:ff***
```

And the other advantage is that it's one less thing to have to specify. And part of the problem there is what to name `fmt::join`? This paper has been using the name `std::format_join`. Is this one of those cases that Bjarne likes to point out as people want more syntax because it's simply novel, or is this one of those cases where the terser syntax is just inscrutable and unnecessary?

I was initially torn on dynamic delimiter, but after spending even a little bit of time working with them in the contexts of this paper, I have become a big fan. I don't actually think `fmt::join` adds anything. In `{fmt}`, formatting ranges wouldn't accept specifiers for each element, so `join` there solved two problems: adding element-specific specifiers and a custom delimiter. But this paper is already expanding the `{fmt}` functionality by allowing specifiers in direct range formatting, adding delimiters there seems in line with that further enhancement.

In fact, we can even go further…

3.3.7.4 Static Delimiter for Ranges

I just showed the idea that we might be able to support:

```
fmt::print("{:ed{}:02x}", mac, ":");  // aa:bb:cc:dd:ee:ff
```

But practically speaking, it is extremely common to know, statically, what the delimiter is. And a lot of the time the delimiter is going to be either empty (`""`) or a single character, as opposed to the default `", "`. In these cases, having a dynamic delimiter seems like pure overhead.

Now the question is, how could we do a static delimiter (i.e. built into the specifier) rather than a dynamic delimiter (i.e. provided as a format argument)? The issue here is we need bounds - the same kinds of bounds we need for pair/tuple. So a starting point might be… let's just use `[]`s. The stuff between the `[]`s is the delimiter:

```
// dynamic delimiter, single colon
fmt::print("{:ed{}:02x}", mac, ":"); // aa:bb:cc:dd:ee:ff

// static delimiter, different amounts of colons
fmt::print("{:ed[:]:02x}", mac);     // aa:bb:cc:dd:ee:ff
fmt::print("{:ed[]:02x}", mac);      // aabbccddeeff
fmt::print("{:ed[::]:02x}", mac);    // aa::bb::cc::dd::ee::ff

// dynamic delimiter, brackets for whatever reason
fmt::print("{:ed{}:02x}", mac, "[]"); // aa[]bb[]cc[]dd[]ee[]ff
```

That is, grammatically, `d{}` or `d{4}` is a dynamic delimiter (referring to the next or 5th argument, respectively), while `d[]` or `d[-]` is a static delimiter (having no delimiter and a single hyphen, respectively). This is easy

enough to parse.

Of course, as the last example illustrates, once we pick some arbitrary brackets for this (and at least in this case we can actually pick square brackets), we run into the problem of: what if the user actually wants to use `]` in their delimiter? Now this makes the specifier much harder to parse or deal with and this quickly becomes the same level of problem as the pair/tuple issue.

This one does have slightly easier solutions, in that we could either:

1. Just not allow `]` in static delimiters, if they want to do that they have to use a dynamic one
2. Go the lua/cmake route and rather than use `[` and `]` to delimit the static delimiter, use `[=[` and `]=]` (except with a variable amount of `=`s, they just have to match).
3. Allow any Unicode open bracket (except `{`), that will then be matched by the corresponding close bracket.

Or, in code form:

```
// option 1)
fmt::print("{:ed{}:02x}", mac, "[]");      // aa[]bb[]cc[]dd[]ee[]ff

// option 2) disambiguate by using ='s
fmt::print("{:ed[=[[]]=]:02x}", mac);      // aa[]bb[]cc[]dd[]ee[]ff
fmt::print("{:ed[==[[]]==]:02x}", mac);    // aa[]bb[]cc[]dd[]ee[]ff

// .. which pessimizes the typical case
fmt::print("{:ed[[:]]:02x}", mac);         // aa:bb:cc:dd:ee:ff

// option 3) use different brackets:
fmt::print("{:ed([]):02x}", mac);          // aa[]bb[]cc[]dd[]ee[]ff
fmt::print("{:ed«[]»:02x}", mac);          // aa[]bb[]cc[]dd[]ee[]ff
fmt::print("{:ed﴿[]﴾:02x}", mac);          // aa[]bb[]cc[]dd[]ee[]ff
```

If we're going to go the route of static delimiter at all, option 1 seems completely sufficient: if you want to use `]` in your delimiter, you have to use dynamic delimiter. That seems like an incredibly rare choice of delimiter anyway, not nearly common enough to either pessimize the overwhelmingly common case in terms of what the specifier string looks like or to overcomplicate what the implementation has to do to make it work.

Using a static delimiter, bounded by `[]`s, does end up being a few characters shorter than using a dynamic delimiter:

```
// format a mac address
fmt::print("{:ed{}:02x}", mac, ":");
fmt::print("{:ed[:]:02x}", mac);

// join words with a space
fmt::print("{:d{}})", words, " ");
fmt::print("{:d[ ]})", words);

// .. or with no delimiter
fmt::print("{:d{}})", words, "");
fmt::print("{:d[]})", words);
```

But the advantage here isn't that we're optimizing for the length of the specifier. The advantage here is that the specifier itself is sufficient to format the argument, so we *only* have to deal with a single argument. I don't care about the four fewer characters. I do care about the one fewer argument and the locality of the delimiter.

There would also be a question of how to implement this. Is a `formatter` allowed to keep a `string_view` into the format specifier ([LWG3651]), to be used in `format`? If we can, then at least this would be a pretty cheap operation. If we can't, that in of itself might be a reason to eschew static delimiters. Note that `{fmt}`'s implementation today does already store `string_views` to the format specifier in order to handle named arguments (which are not yet standardized), which at least suggests that this is a safe thing to do - although this should probably be clarified in the `formatter` requirements regardless of whether we pursue static delimiters (since just because we don't in this context, doesn't mean that users won't want to for their own types).

A more complete example from my own code base, where in some contexts we have a type like `span<unsigned char>` we want to print in both hex and ascii. The three different levels of functionality there are:

```
// use fmt::join
fmt::print("{:#04x}: \"{}\"\n",
    fmt::join(data, ","),
    fmt::join(data | views::transform([](unsigned char c){
        return std::isprint(c) ? (char)c : '.';
    }), ""));

// use dynamic delimiter
fmt::print("{:d{}:#04x} \"{:ed{}:}\"\n",
    data,
    ",",
    data | views::transform([](unsigned char c){
        return std::isprint(c) ? (char)c : '.';
    }),
    "");

// use static delimiter
fmt::print("{:d[,]:#04x} \"{:ed[]:}\"\n",
    data,
    data | views::transform([](unsigned char c){
        return std::isprint(c) ? (char)c : '.';
    }));
```

Although with this particular example, this paper provides a better way to print the second part of this. We're producing a range of `char` and we want to print it with no delimiter and quoted. That's `{:s}`. So really the right way to present these levels are:

```
// use fmt::join
fmt::print("{:#04x}: {:s}\n",
    fmt::join(data, ","),
    data | views::transform([](unsigned char c){
        return std::isprint(c) ? (char)c : '.';
    }));

// use dynamic delimiter
fmt::print("{:d{}:#04x} {:s}\n",
    data,
    ",",
    data | views::transform([](unsigned char c){
        return std::isprint(c) ? (char)c : '.';
    }));

// use static delimiter
fmt::print("{:d[,]:#04x} {:s}\n",
    data,
    data | views::transform([](unsigned char c){
```

```
        return std::isprint(c) ? (char)c : '.';
    }));
```

Static delimiter is limited by the fact that the delimiter must be static, so it cannot be the whole solution the problem. But it *is* a good solution to the common case where the delimiter is statically known. When it's not (or based on user preference or other considerations), dynamic delimiter will be available as a fallback. Between these two options, that covers the complete set of functionality that `{fmt}` provides under `fmt::join` (in fact, more than complete).

3.3.7.5 Pair and Tuple Specifiers

This is the hard part.

To start with, we for consistency will support the same fill/align/width specifiers as usual.

For ranges, we can have the underlying element's `formatter` simply parse the whole format specifier string from the character past the `:` to the `}`. The range doesn't care anymore at that point, and what we're left with is a specifier that the underlying element should understand (or not).

But for `pair`, it's not so easy, because format strings can contain *anything*. Absolutely anything. So when trying to parse a format specifier for a `pair<X, Y>`, how do you know where `X`'s format specifier ends and `Y`'s format specifier begins? This is, in general, impossible.

In [P2286R3], this paper used Tim's insight to take a page out of `sed`'s book and rely on the user providing the specifier string to actually know what they're doing, and thus provide their own delimiter. `pair` will recognize the first character that is not one of its formatters as the delimiter, and then delimit based on that. This previous revision had proposed the following:

Format String	Contents	Formatted Output
{}	pair(10, 1729)	(10, 1729)
{:}	pair(10, 1729)	(10, 1729)
{::#x:04X}	pair(10, 1729)	(0xa, 06C1)
{:|#x|04X}	pair(10, 1729)	(0xa, 06C1)
{:Y#xY04X}	pair(10, 1729)	(0xa, 06C1)

The last three rows are equivalent, the difference is which character is used to delimit the specifiers: `:` or `|` or `Y`.

This approach, while technically functional, still leaves something to be desired. For one thing, these examples are already difficult to read and I haven't even shown any additional nesting. We're using to nested parentheses, brackets, or braces, but there's nothing visually nested here. And it's not even clear how to do something like that anyway. Several people expressed a desire to have a delimiter language that at least has some concept of nesting built-in - such as naturally-nesting punctuation like`()`s, `[]`s, or `{}`s (Unicode has plenty of other pairs of open/close characters. I could revisit my Russian roots with « and », or use something prettier like ⦑ and ⦒).

The point, ultimately, is that it is difficult to comme up with a format specifier syntax that works *at all* in the presence of types that can use arbitrary characters in their specifiers. Like formatting `std::chrono::system_clock::now()`:

Format String	Formatted Output
{}	2021-10-24 20:33:37

Format String	Formatted Output
`{:%Y-%m-%d}`	`2021-10-24`
`{:%H:%M:%S}`	`20:33:37`
`{:%H hours, %M minutes, %S seconds}`	`20 hours, 33 minutes, 37 seconds`

Because there is reasonable concern about the complexity of the initially proposed solution, and because there doesn't seem to be a lot of demand for actually being able to do this, in contrast to the very clear and present demand of being able to format pairs and tuples simply by default - this revision of this paper is withdrawing this part of the proposal in an effort to get the rest of the paper in for C++23.

To summarize: `std::pair` and `std::tuple` will only support:

— the fill/align/width specifiers from *std-format-spec*

— the `?` specifier, to format as debug (which is a no-op, since it will always format as debug, since there is no opt-out provided)

— the `m` specifier, only valid for `pair` or 2-tuple, to format as `k: v` instead of `(k, v)`

It will additionally provide the function:

```
void set_debug_format();
```

and `pair` and `tuple` will provide the function:

```
void set_map_format();
```

which for `tuple` of size other than 2 will throw an exception (since you cannot format those as a map). To clarify the map specifier:

Format String	Contents	Formatted Output
`{}`	`pair(1, 2)`	`(1, 2)`
`{:m}`	`pair(1, 2)`	`1: 2`
`{:m}`	`tuple(1, 2)`	`1: 2`
`{}`	`tuple(1)`	`(1)`
`{:m}`	`tuple(1)`	exception or compile error
`{}`	`tuple(1, 2, "3"s)`	`(1, 2, "3")`
`{:m}`	`tuple(1, 2, "3"s)`	exception or compile error

§ 3.3.8 Escaping Behavior

Escaping of a string in a Unicode encoding is done by translating each UCS scalar value, or a code unit if it is not a part of a valid UCS scalar value, in sequence:

— If a UCS scalar value is one of `'\t'`, `'\r'`, `'\n'`, `'\\'` or `'"'`, it is replaced with `"\\t"`, `"\\r"`, `"\\n"`, `"\\\\"` and `"\\\""` respectively.

— Otherwise, if a UCS scalar value has a Unicode property Separator (Z) or Other (C), it is replaced with its universal character name escape sequence in the form `"\\u{`*`simple-hexadecimal-digit-sequence`*`}"` as proposed by [P2290R2], where *simple-hexadecimal-digit-sequence* is a hexadecimal representation of the UCS scalar value without leading zeros.

— Otherwise, if a UCS scalar value has a Unicode property Grapheme_Extend and there are no UCS scalar values preceding it in the string without this property, it is replaced with its universal character name escape sequence as above.

— Otherwise, a code unit that is not a part of a valid UCS scalar value is replaced with a hexadecimal escape sequence in the form `"\\x{simple-hexadecimal-digit-sequence}"` as proposed by [P2290R2], where *simple-hexadecimal-digit-sequence* is a hexadecimal representation of the code unit without leading zeros.

— Otherwise, a UCS scalar value is copied as is.

The same applies to wide strings with `'...'` and `"..."` replaced with `L'...'` and `L"..."` respectively.

For non-Unicode encodings an implementation-defined equivalent of Unicode properties is used.

Escape rules for characters are similar except that `'\''` is escaped instead of `'"'` and `'"'` is not escaped.

Examples:

```
std::cout << std::format("{:?}", std::string("h\tllo"));
// Output: "h\tllo"

std::cout << std::format("{:?}", std::string("\0 \n \t \x02 \x1b", 9));
// Output: "\{0} \n \t \x{2} \x{1b}"

std::cout << std::format("{:?}, {:?}, {:?}", " \" ' ", '"', '\'');
// Output: " \" ' ", '"', '\''

std::cout << std::format("{:?}", "\xc3\x28"); // invalid UTF-8
// Output: "\x{c3}\x{28}"

std::cout << std::format("{:?}", "\u0300"); // assuming a Unicode encoding
// Output: "\u{300}"
// (as opposed to "̀" with an accent on the first ")

auto s = std::format("{:?}", "Привет, 🕯!"); // assuming a Unicode encoding
// s == "\"Привет, 🕯!\""
```

Notes:

— SG16 requested using the escape sequence format proposed by [P2290R2], `{fmt}` uses a non-braced escape format (same as Python).

— Grapheme_Extend part is not implemented in `{fmt}` yet.

§ 3.3.9 Examples with user-defined types

Let's say a user has a type like:

```
struct Foo {
    int bar;
    std::string baz;
};
```

And want to format `Foo{.bar=10, .baz="Hello World"}` as the string `Foo(bar=10, baz="Hello World")`. They can do so this way:

```
template <>
struct formatter<Foo, char> {
```

```
    template <typename FormatContext>
    constexpr auto format(Foo const& f, FormatContext& ctx) const {
        return format_to(ctx.out(), "Foo(bar={}, baz={:?})", f.bar, f.baz);
    }
};
```

How about wrappers?

Let's say you have your own implementation of `Optional`, that you want to format the same way that Rust does: so that a disengaged one formats as `None` and an engaged one formats as `Some(??)`. We can start by:

```
template <formattable<char> T>
struct formatter<Optional<T>, char> {
    // we'll skip parse for now

    template <typename FormatContext>
    auto format(Optional<T> const& opt, FormatContext& ctx) {
        if (not opt) {
            return format_to(ctx.out(), "None");
        } else {
            return format_to(ctx.out(), "Some({})", *opt);
        }
    }
};
```

If we had an `Optional<string>("hello")`, this would format as `Some(hello)`. Which may be fine. But what if we wanted to format it as `Some("hello")` instead? That is, take advantage of the quoting rules described earlier. What do you write instead of `*opt` to format `string`s (or `char`s or user-defined string-like types) as quoted in this context?

We can both add support for quoting/escaping and also arbitrary specifiers at the same time:

```
template <formattable<char> T>
struct formatter<Optional<T>, char> {
    formatter<T, char> underlying;

    template <typenaem ParseContext>
    constexpr auto parse(ParseContext& ctx) {
        auto end = underlying.parse(ctx);
        if constexpr (requires { underlying.set_debug_format(); }) {
            underlying.set_debug_format();
        }
        return end;
    }

    template <typename FormatContext>
    auto format(Optional<T> const& opt, FormatContext& ctx) {
        if (not opt) {
            return format_to(ctx.out(), "None");
        } else {
            ctx.advance_to(format_to(ctx.out(), "Some("));
            auto out = underlying.format(*opt, ctx);
            *out++ = ')';
            return out;
        }
    }
};
```

This lets me format `Optional<string>("hello")` as `Some("hello")` by default, or format `Optional<int>(42)` as `Some(0x2a)` if I provide the specifier string `"{:#x}"`.

3.4 Implementation Challenges

I implemented the range and pair/tuple portions of this proposal on top of libfmt. I chose to do it on top so that I can easily share the implementation [fmt-impl], as such I could not implement `?` support for strings and char, though that is not a very interesting part of this proposal (at least as far as implementability is concerned). There were two big issues that I ran into that are worth covering.

3.4.1 Wrapping `basic_format_context` is not generally possible

In order to be able to provide an arbitrary type's specifiers to format a range, you have to have a `formatter<V>` for the underlying type and use that specific `formatter` in order to `parse` the format specifier and then `format` into the given context. If that's all you're doing, this isn't that big a deal, and I showed a simplified implementation of `range_formatter<V>` earlier.

However, if you additionally want to support fill/pad/align, then the game changes. You can't format into the provided context - you have to format into *something else* first and then do the adjustments later. Adding padding support ends up doing something more like this:

No padding

```
template <typename R, typename
        FormatContext>
constexpr auto format(R&& r,
        FormatContext& ctx) {
    auto out = ctx.out();
    *out++ = '[';
    auto first = std::ranges::begin(r);
    auto last = std::ranges::end(r);
    if (first != last) {
        ctx.advance_to(std::move(out));
        out = underlying.format(*first,
        ctx);
        for (++first; first != last;
        ++first) {
            *out++ = ',';
            *out++ = ' ';

        ctx.advance_to(std::move(out));
            out =
        underlying.format(*first, ctx);
        }
    }
    *out++ = ']';
    return out;
}
```

With padding

```
template <typename R, typename FormatContext>
constexpr auto format(R&& r, FormatContext& ctx)
        {
    // fmt has a dynamically growing buffer:
        memory_buffer
    // and a type-erased iterator into it:
        appender
    fmt::memory_buffer buf;
    fmt::basic_format_context<fmt::appender,
        char>
      bctx(fmt::appender(buf), ctx.args(),
        ctx.locale());

    auto out = bctx.out();
    *out++ = '[';
    auto first = std::ranges::begin(r);
    auto last = std::ranges::end(r);
    if (first != last) {
        bctx.advance_to(std::move(out));
        out = underlying.format(*first, bctx);
        for (++first; first != last; ++first) {
            *out++ = ',';
            *out++ = ' ';
            bctx.advance_to(std::move(out));
            out = underlying.format(*first,
        bctx);
        }
    }
    *out++ = ']';

    // at this point, we formatted our range into
        buf, so
```

No padding	With padding

	`// now we need to format buf into the *real* context,`
	`// ctx.out(), with fill/pad/align. That part isn't`
	`// interesting for our purposes here`
	`return write-padded-aligned(ctx.out(), buf);`
	`}`

It's mostly the same - we format into `bctx` instead of `ctx` and then `write` into `ctx` later using the `specs` that we already parsed. The code seems straightforward enough, except…

First, we don't even expose a way to construct `basic_format_context` so can't do this at all (there's no specified constructor for it in 20.20.6.4 [format.context]). Nor do we expose a way of constructing an iterator type for formatting into some buffer. And if we could construct these things, the real problem hits when we try to construct this new context. We need some kind of `fmt::basic_format_context<???, char>`, and we need to write into some kind of dynamic buffer, so `fmt::appender` is the appropriate choice for iterator. But the issue here is that `fmt::basic_format_context<Out, CharT>` has a member `fmt::basic_format_args<basic_format_context>` - the underlying arguments are templates *on the context*. We can't just… change the `basic_format_args` to have a different context, this is a fairly fundamental attachment in the design.

The *only* type for the output iterator that I can support in this implementation is precisely `fmt::appender`.

This seems like it'd be *extremely* limiting.

Except it turns out that `{fmt}` uses exactly this iterator in a whole lot of places. `fmt::print`, `fmt::format`, `fmt::format_to`, `fmt::format_to_n`, `fmt::vformat`, etc., all only use this one iterator type. This is because of [P2216R3]'s efforts to reduce code bloat by type erasing the output iterator.

However, there is one part of `{fmt}` that uses a different iterator type, which the above implementation fails on:

```
fmt::format("{:::d}", vector{vector{'a'}, vector{'b', 'c'}});             // ok:
        [[97], [98, 99]]
fmt::format(FMT_COMPILE("{:::d}"), vector{vector{'a'}, vector{'b', 'c'}}); // ill-
        formed
```

The latter fails because there the initial output iterator type is `std::back_insert_iterator<std::string>`. This is a different iterator type from `fmt::appender`, so we get a mismatch in the types of the `basic_format_args` specializations, and cannot compile the construction of `bctx`.

This can be worked around (I just need to know what the type of the buffer needs to be, in the usual case it's `fmt::memory_buffer` and here it becomes `std::string`, that's fine), but it means we really need to nail down what the requirements of the `formatter` API are. One of the things we need to do in this paper is provide a `formattable` concept. From a previous revision of that paper, dropping the `char` parameter for simplicity, that looks like:

```
template <class T>
concept formattable-impl =
    std::semiregular<fmt::formatter<T>> &&
    requires (fmt::formatter<T> f,
              const T t,
              fmt::basic_format_context<char*, char> fc,
```

```
                fmt::basic_format_parse_context<char> pc)
    {
        { f.parse(pc) } ->
        std::same_as<fmt::basic_format_parse_context<char>::iterator>;
        { f.format(t, fc) } -> std::same_as<char*>;
    };

template <class T>
concept formattable = formattable-impl<std::remove_cvref_t<T>>;
```

Note that based on the resolution of [LWG3636], the call to `format` may be on a `const fmt::formatter<T>` instead.

I use `char*` as the output iterator, but my `range_formatter<V>` cannot support `char*` as an output iterator type at all. Do `formatter` specializations need to support any output iterator type? If so, how can we implement fill/align/pad support in `range_formatter`?

The simplest approach would be to state that there actually is only one output iterator type that need be support per character type. But this would prohibit the approach `{fmt}` uses to process the format string at compile time, as well as any potential future optimizations. This just seems like a non-starter.

A different approach would be to introduce a new API that allows the implementation to produce a new context for us. That approach could look like this:

```
template <typename V, typename FormatContext>
constexpr auto format(V&& value, FormatContext& ctx) -> typename
        FormatContext::iterator
{
    // ctx here is a basic_format_context<OutIt, CharT>, for some output iterator
    // and some character type

    // can use a vector<CharT>, basic_string<CharT>, or some custom buffer like
    // fmt::buffer, user's choice
    vector<CharT> buf;

    // The job of the retargeted_format_context class template is to produce
    // a new specialization of basic_format_context for the provided iterator
    // that simply does The Right Thing (TM).
    // We do not need bctx here to be specifically (w)format_context, just some
    // specialization of basic_format_context that is definitely going to write
    // into buf (regardless of buf's type).
    retargeted_format_context rctx(ctx, std::back_inserter(buf));
    auto& bctx = rctx.context();

    // format into bctx...
}
```

There is one fundamental limitation here that is sort of inherent in the design. If the user-defined types want to reference some other argument (i.e. something like dynamic width or dynamic precision) but want that other argument to *also* be a user-defined type (rather than just an integer or `string_view`/`char const*`), they basically cannot. Thta's not an option. User-defined types are type erased as `handle` (see 20.20.7.1 [format.arg]), and `handle` can only be formatted with a `(w)format_parse_context` - which only the implementation would have access to.

However, if we ignore user-defined types entirely, it is straightforward to convert all the other `format_args` from one context to another, since we know everything about all of those types and they are all cheap to copy.

The implementation approach I used is as follows:

```
// effectively a tagged version of fmt::appender, solely for
// specializing on top of
template <typename Old>
struct custom_appender : appender {
    using appender::appender;
};

// specialization of basic_format_args for use with custom_appender
// This ended up being easier than specializing basic_format_context.
// This specialization is only used in the context of retargeted_format_context.
//
// Note that here we hold a reference to the original basic_format_args: we don't
// have to make a copy, and we only produce a new basic_format_arg if actually
// requiried by the formatting. This means we don't have to pay for anything that
// we don't use
template <typename Old>
struct basic_format_args<basic_format_context<custom_appender<Old>, char>> {
    using old_args = basic_format_args<basic_format_context<Old, char>>;
    using new_context = basic_format_context<custom_appender<Old>, char>;
    using format_arg = basic_format_arg<new_context>;
    using size_type = int;

    old_args const& orig_args;

    basic_format_args(old_args const& orig)
        : orig_args(orig)
    { }

    constexpr auto get(int id) const -> format_arg {
        return visit_format_arg([]<typename T>(T const& arg) -> format_arg {
            // User-defined types or out-of-range arguments can't produce any
            // valid format_arg, so we return no format_arg
            if constexpr (std::same_as<T, typename old_args::format_arg::handle>
                        or std::same_as<T, monostate>) {
                return format_arg();
            } else {
                // ... but for all the other types, this is a cheap copy
                // T is bool, char, some integral type, some floating point
                // type, char const*, string_view, or void const*
                return detail::make_arg<new_context>(arg);
            }
        }, orig_args.get(id));
    }

    // These next two functions are {fmt}-specific, since std:: doesn't
    // have argument names. But if it did, as you can see, these calls are
    // pretty straightforward
    constexpr auto get(fmt::string_view name) const -> format_arg {
        int id = orig_args.get_id(name);
        return id >= 0 ? get(id) : format_arg();
    }

    constexpr auto get_id(fmt::string_view name) const -> int {
        return orig_args.get_id(name);
    }
};
```

```cpp
// In the case where we do need to retarget, we build a new context using
// custom_appender<Context::iterator>, which will use the specialization of
// basic_format_args defined above (no new basic_format_args are created)
template <typename Context, typename OutputIt>
struct retargeted_format_context {
    detail::iterator_buffer<OutputIt, char> buffer;

    using iterator = custom_appender<typename Context::iterator>;
    using new_context = basic_format_context<iterator, char>;
    new_context erased_ctx;

    retargeted_format_context(Context& ctx, OutputIt it)
        : buffer(it)
        , erased_ctx(iterator(buffer),
                        basic_format_args<new_context>(ctx.args()),
                        ctx.locale())
    { }

    auto context() -> new_context& {
        return erased_ctx;
    }

    // in fmt, this iterator is buffered, so we need to flush it
    void flush() {
        (void)buffer.out();
    }
};

// In the "happy" case (i.e. we're just using fmt::print), we don't need to do
// any of this, the args are already the correct type so copying them is fine.
// All we need to do is create a new context
template <typename CharT, typename OutputIt>
struct retargeted_format_context<basic_format_context<OutputIt, CharT>, OutputIt>
{
    basic_format_context<OutputIt, CharT> ctx;

    retargeted_format_context(basic_format_context<OutputIt, CharT>& ctx, OutputIt
        it)
        : ctx(it, ctx.args(), ctx.locale())
    { }

    auto context() -> basic_format_context<OutputIt, CharT>& { return ctx; }

    void flush() { }
};
```

You can see this in the implementation I shared [fmt-impl], on lines 65-140.

We don't strictly need to provide `retargeted_format_context` just to format ranges (the implementation would do something like this internally). But if users want to be able to solve this problem (e.g. fill/pad/align for a user-defined type where all you have is `formatter<T>` for unknown `T`) for any of their own types, they'll need to do something like this as well, so this functionality should be provided to let them do that.

3.4.2 Manipulating `basic_format_parse_context` to search for sentinels

Even though this paper is no longer proposing complex `pair` and `tuple` support, it's still useful to discuss one of the examples that could have been supported:

```
fmt::format("{:|#x|*^10}", std::pair(42, "hello"s));
```

In order for this to work, the `formatter<int>` object needs to be passed a context that just contains the string `"#x"` and the `formatter<string>` object needs to be passed a context that just contains the string `"*^10"` (or possibly `"*^10}"`). This is because `formatter<T>::parse` must consume the whole context. That's the API.

But `basic_format_parse_context` does not provide a way for you to take a slice of it, and we can't just construct a new object because of the dynamic argument counting support. Not just *any* context, but *specifically that one*.

Tim's suggested design for how to even do specifiers for `pair` also came with a suggested implementation: use a `sentry`-like type that temporarily modifies the context and restores it later. The use of this type looks like this:

```
auto const delim = *begin++;
ctx.advance_to(begin);
tuple_for_each_index(underlying, [&](auto I, auto& f){
    auto next_delim = std::find(ctx.begin(), end, delim);
    if constexpr (I + 1 < sizeof...(Ts)) {
        if (next_delim == end) {
            throw fmt::format_error("ran out of specifiers");
        }
    }

    end_sentry _(ctx, next_delim);
    auto i = f.parse(ctx);
    if (i != next_delim && *i != '}') {
        throw fmt::format_error("this is broken");
    }

    if (next_delim != end) {
        ++i;
    }
    ctx.advance_to(i);
});
```

This ensures that each element of the `pair`/`tuple` only sees its part of the whole parse string, which is the only part that it knows what to do anything with.

Without something like this in the library, it'd be impossible to do this sort of complex specifier parsing. You could support ranges (there, we only have one underlying element, so it parses to the end), but not pair or tuple. We *could* say that since pair and tuple are library types, the library should just Make This Work, but there are surely other examples of wanting to do this sort of thing and it doesn't feel right to not allow users to do it too.

As with `retargeted_format_context`, if we adopted the `pair`/`tuple` specifiers design, we wouldn't have to expose something like this in the standard library. The implementation would need to do it internally and it could do whatever it needs to do to get it done. But it's still useful functionality to be able to export to users. And especially if we're not going to adopt arbitrary pair/tuple specifiers, I think it's important to give users the tools to experiment with them.

This design space is, thankfully, slightly easier than the previous problem: this is basically what you have to do. Not much choice, I don't think.

§ 3.4.3 Parsing of alignment, padding, and width

The first two issues in this section are serious implementation issues that require design changes to `<format>`. This one doesn't *require* changes, and this paper won't propose changes, but it's worth pointing out nevertheless. Alignment, padding, and width are the most common and fairly universal specifiers. But we don't provide a public API to actually parse them.

When implementing this in `fmt`, I just took advantage of `fmt`'s implementation details to make this a lot easier for myself: a type (`dynamic_format_specs<char>`) that holds all the specifier results, a function that understands those to let you write a padded/aligned string (`write`), and several parsing functions that are well designed to do the right thing if you have a unique set of specifiers you wish to parse (the appropriately-named `parse_align` and `parse_width`).

These don't have to be standardized, as nothing in these functions is something that a user couldn't write on their own. And this paper is big enough already, so it, again, won't propose anything in this space. But it's worth considering for the future.

§ 3.5 How to support those views which are not `const`-iterable?

In a previous revision of this paper, this was a real problem since at the time `std::format` accepted its arguments by `const Args&...`

However, [P2418R2] was speedily adopted specifically to address this issue, and now `std::format` accepts its arguments by `Args&&...` This allows those views which are not `const`-iterable to be mutably passed into `format()` and `print()` and then mutably into its formatter. To support both `const` and non-`const` formatting of ranges without too much boilerplate, we can do it this way:

```
template <formattable V>
struct range_formatter {
    template <typename ParseContext>
    constexpr auto parse(ParseContext&);

    template <range R, typename FormatContext>
        requires same_as<remove_cvref_t<range_reference_t<R>>, V>
    constexpr auto format(R&&, FormatContext&);
};

template <range R> requires formattable<range_reference_t<R>>
struct formatter<R> : range_formatter<range_reference_t<R>>
{ };
```

`range_formatter` allows reducing unnecessary template instantiations. Any range of `int` is going to `parse` its specifiers the same way, there's no need to re-instantiate that code n times. Such a type will also help users to write their own formatters, since they can have a member `range_formatter<int>` to handle any range of `int` (or `int&` or `int const&`) rather than having to have a specific `formatter<my_special_range>`.

§ 3.6 What additional functionality?

There's three layers of potential functionality:

1. Top-level printing of ranges: this is `fmt::print("{}", r);`
2. A format-joiner which allows providing a a custom delimiter: this is provided in `{fmt}` under the spelling `fmt::print("{:02x}", fmt::join(r, ":"))`. Previous revisions of the paper sought to simply

standardize this under the name `std::format_join`, but this paper has since evolved to both allow custom specifier directly to format `r` as well as now providing the ability to directly provide the delimiter. A `fmt::join`-like facility is thus not necessary and not proposed.

3. A more involved version of a format-joiner which takes a delimiter and a callback that gets invoked on each element. fmt does not provide such a mechanism, though the Rust itertools library does:

```
let matrix = [[1., 2., 3.],
              [4., 5., 6.]];
let matrix_formatter = matrix.iter().format_with("\n", |row, f| {
                                f(&row.iter().format_with(", ", |elt, g| g(&elt)))
                            });
assert_eq!(format!("{}", matrix_formatter),
           "1, 2, 3\n4, 5, 6");
```

Even this example is also already solvable with the facilities suggested in this revision, as `format("{:ed[\n]:e}", matrix)` (or the `"\n"` delimiter can be provided dynamically). The one piece of flexibility *not* provided in this revision is, in the case of formatting a range of ranges, there is currently no ability to provide a custom bracket to the inner range. You either get the default `[]`s or you can get nothing, but you have no way of providing, say… `()`s or `<>` or ⦇⦈s or whatever. This would have to be provided by either the user writing a custom formatter for their custom type, or a future extension of this paper which explores how to do custom brackets.

But given the wealth of functionality that is available, that's pretty great.

3.6.1 `fmt::join`

If we were not going to support dynamic and static delimiters for ranges, then we need some other mechanism to provide a custom delimiter. That mechanism exists in `{fmt}` already under the name `fmt::join`.

It works like this:

Format String	Contents	Formatted Output
{}	`fmt::join(std::vector{1, 2, 3}, ", ")`	1, 2, 3
[{}]	`fmt::join(std::vector{1, 2, 3}, ", ")`	[1, 2, 3]
[{}]	`fmt::join(std::vector{1, 2, 3}, "--")`	[1--2--3]
[{}]	`fmt::join(std::vector{1, 2, 3}, "--"s)`	[1--2--3]
{:x}	`fmt::join(std::vector{10, 20, 30}, ":")`	a:14:1e
{:#04X}	`fmt::join(std::vector{10, 20, 30}, ":")`	0X0A:0X14:0X1E
{}	`fmt::join(std::vector{"h\tllo"s, "world"s}, ", ")`	h llo, world
{:?}	`fmt::join(std::vector{"h\tllo"s, "world"s}, ", ")`	"h\tllo", "world"

`std::format_join` (since we already have a `std::views::join` and none of the formatting is in a `fmt` namespace) will accept a `viewable_range` of `formattable` (based on the range's `reference` type) and a delimiter which is convertible to `(w)string_view`, and produce an `std::`*`format-join-view`* object. That object will take as a specifier whatever the underlying type accepts, and use that result to format each element, using the provided delimiter. Unlike the default ranges formatter, strings and chars are not printed escaped/quoted: users need to provide `?` for that functionality.

Note that `std::format_join` does not (and cannot) support pad/align/width. But some people might prefer reading `join(r, "-")` in code over something like `d{}` with a `"-"` somewhere or `d[-]`. For those people, it is pretty straightforward to implement `fmt::join`, and that implementation is provided here (even though the paper is not proposing that we standardize this facility, because I've become convinced at this point that it is strictly worse than the static/dynamic delimiter approach that is proposed in this facility).

```
template <std::ranges::input_range V>
    requires std::ranges::view<V>
          && formattable<std::ranges::range_reference_t<V>>
struct format_join_view {
    V v;
    fmt::string_view delim;
};

template <std::ranges::input_range V>
struct fmt::formatter<format_join_view<V>> {
    fmt::formatter<std::remove_cvref_t<std::ranges::range_reference_t<V>>>
        underlying;

    template <typename ParseContext>
    constexpr auto parse(ParseContext& ctx) {
        return underlying.parse(ctx);
    }

    template <typename R, typename FormatContext>
    constexpr auto format(R&& r, FormatContext& ctx) {
        auto it = std::ranges::begin(r.v);
        auto out = ctx.out();
        if (it != std::ranges::end(r.v)) {
            out = underlying.format(*it, ctx);
            for (++it; it != std::ranges::end(r.v); ++it) {
                ctx.advance_to(std::ranges::copy(r.delim, out).out);
                out = underlying.format(*it, ctx);
            }
        }
        return out;
    }
};

template <std::ranges::viewable_range R>
    requires formattable<std::ranges::range_reference_t<R>>
auto format_join(R&& r, fmt::string_view delim) {
    return format_join_view{std::views::all(std::forward<R>(r)), delim};
}
```

§ 3.7 `format` or `std::cout`?

Just `format` is sufficient.

§ 3.8 What about `vector<bool>`?

Nobody expected this section.

The `value_type` of this range is `bool`, which is formattable. But the `reference` type of this range is `vector<bool>::reference`, which is not. In order to make the whole type formattable, we can either make `vector<bool>::reference` formattable (and thus, in general, a range is formattable if its `reference` types is

formattable) or allow formatting to fall back to constructing a `value_type` for each `reference` (and thus, in general, a range is formattable if either its `reference` type or its `value_type` is formattable).

For most ranges, the `value_type` is `remove_cvref_t<reference>`, so there's no distinction here between the two options. And even for `zip` [P2321R2], there's still not much distinction since it just wraps this question in tuple since again for most ranges the types will be something like `tuple<T, U>` vs `tuple<T&, U const&>`, so again there isn't much distinction.

`vector<bool>` is one of the very few ranges in which the two types are truly quite different. So it doesn't offer much in the way of a good example here, since `bool` is cheaply constructible from `vector<bool>::reference`. Though it's also very cheap to provide a formatter specialization for `vector<bool>::reference`.

Rather than having the library provide a default fallback that lifts all the `reference` types to `value_types`, which may be arbitrarily expensive for unknown ranges, this paper proposes a format specialization for `vector<bool>::reference`. This type is actually defined as `vector<bool, Alloc>::reference`, so the wording for this aspect will be a little awkward (we'll need to provide a type trait *`is-vector-bool-reference`*`<R>`, etc., but this is a problem for the wording and the implementation to deal with).

§ 4 Proposal

The standard library will provide the following utilities:

- A `formattable` concept.
- A `range_formatter<V>` that uses a `formatter<V>` to `parse` and `format` a range whose `reference` is similar to `V`. This can accept a specifier on the range (align/pad/width as well as string/map/debug/empty/static delimiter/dynamic delimiter) and on the underlying element (which will be applied to every element in the range).
- A `tuple_formatter<tuple<Ts...>>` that uses a `formatter<T>` for each `T` in `Ts...` to `parse` and `format` either a `pair`, `tuple`, or `array` with appropriate elements. This can accept a specifier on the tuple-like (align/pad/width as well as map/static delimiter/dynamic delimiter), but will not accept any specifier the underlying elements.
- A `retargeted_format_context` facility that allows the user to construct a new `(w)format_context` with a custom output iterator.
- An `end_sentry` facility that allows the user to manipulate the parse context's range, for generic parsing purposes (so that users can, if they want, write their own arbitrarily-complex pair/tuple formatting).

The standard library should add specializations of `formatter` for:

- any type `R` that is a `range` whose `reference` is `formattable`, which inherits from `range_formatter<remove_cvref_t<ranges::range_reference_t<R>>>`
- `pair<T, U>` if `T` and `U` are `formattable`, which inherits from `tuple_formatter<tuple<remove_cvref_t<T>, remove_cvref_t<U>>>`
- `tuple<Ts...>` if all of `Ts...` are `formattable`, which inherits from `tuple_formatter<tuple<remove_cvref_t<Ts>...>>`

Note that the `pair` and `tuple` formatters both inherit from `tuple_formatter<tuple<Ts...>>`. This is to keep the pattern of defaulting the `charT` parameter as the second parameter, which otherwise would have to be flipped and

look exceedingly awkward.

Additionally, the standard library should provide the following more specific specializations of `formatter`:

— `vector<bool, Alloc>::reference` (which formats as a `bool`)

— all the associative maps (`map`, `multimap`, `unordered_map`, `unordered_multimap`) if their respective key/value types are `formattable`. This accepts the same set of specifiers as any other range, except by *default* it will format as `{k: v, k: v}` instead of `[(k, v), (k, v)]`

— all the associative sets (`sets`, `multiset`, `unordered_set`, `unordered_multiset`) if their respective key/value types are `formattable`. This accepts the same set of specifiers as any other range, except by *default* it will format as `{v1, v2}` instead of `[v1, v2]`

Formatting for `string`, `string_view`, and `char`/`wchar_t` will gain a `?` specifier, which causes these types to be printed as escaped and quoted if provided. Ranges and tuples will, by default, print their elements as escaped and quoted, unless the user provides a specifier for the element.

§ 4.1 Wording

This wording is very much incomplete, in the interests of time to try to get this paper in C++23.

The wording here is grouped by functionality added rather than linearly going through the standard text.

§ 4.1.1 Concept `formattable`

First, we need to define a user-facing concept. We need this because we need to constrain `formatter` specializations on whether the underlying elements of the `pair`/`tuple`/range are formattable, and users would need to do the same kind of thing for their types. This is tricky since formatting involves so many different types, so this concept will never be perfect, so instead we're trying to be good enough.

Change 20.20.1 [format.syn]:

```
namespace std {
  // ...
  // [format.formatter], formatter
  template<class T, class charT = char> struct formatter;

  // [format.parse.ctx], class template basic_format_parse_context
  template<class charT> class basic_format_parse_context;
  using format_parse_context = basic_format_parse_context<char>;
  using wformat_parse_context = basic_format_parse_context<wchar_t>;
+ // [format.formattable], formattable
+ template<class T, class charT>
+   concept formattable = see below;
  // ...
}
```

Add a clause [format.formattable] under 20.20.6 [format.formatter] and likely after 20.20.6.1 [formatter.requirements]:

1 Let *fmt-iter-for*<charT> be an implementation-defined type that models output_iterator<const charT&> ([iterator.concept.output]).

```
template<class T, class charT>
concept formattable-impl =
    semiregular<formatter<T, charT>> &&
    requires (formatter<T, charT> f,
              const formatter<T, charT> cf,
              T t,
              basic_format_context<fmt-iter-for<charT>, charT> fc,
              basic_format_parse_context<charT> pc) {
        { f.parse(pc) } -> same_as<basic_format_parse_context<charT>::iterator>;
        { cf.format(t, fc) } -> same_as<fmt-iter-for<charT>>;
    };

template<class T, class charT>
concept formattable = formattable-impl<remove_cvref_t<T>, charT>;
```

2 A type `T` and a character type `charT` model `formattable` if `formatter<T, charT>` meets the *Formatter* requirements ([formatter.requirements]).

§ 4.1.2 Retargeting `format_context`

Add to… somewhere:

```
// [format.retargeted.context]
template<class Context, class OutputIt>
 struct retargeted_format_context;
```

And:

1 `retargeted_format_context` creates a new `basic_format_context` to allow for formatting into a custom buffer. [*Note*: This allows a `formatter` to change the output that a different `formatter` produces, for instance to add alignment or padding. *-end note*]

2 [*Example*:

```
struct NoCapes {
    string_view value;
};

template <>
struct formatter<NoCapes> {
    formatter<string_view> fmt;

    template <class ParseContext>
    constexpr auto parse(ParseContext& ctx) {
        return fmt.parse(ctx);
    }

    template <class FormatContext>
    auto format(NoCapes nc, FormatContext& ctx) const {
        vector<char> edna;
        retargeted_format_context new_ctx(ctx, back_inserter(edna));
        fmt.format(nc.value, new_ctx.context());
        new_ctx.flush();

        erase_if(edna, [](char c){
            constexpr string_view capes = "capes";
            return capes.contains(c);
        });
```

```
        return copy(edna.begin(), edna.end(), ctx.out());
    }
};

print("'{}'\n", NoCapes{"scathing concession"}); // prints 'thing onion'
```

-end example]

And:

```
template<class Context, class OutputIt>
 class retargeted_format_context {
   using RetargetIt = unspecified;
   using NewContext = basic_format_context<RetargetIt, typename
        Context::char_type>; // exposition only
   NewContext new_context_;
        // exposition only

 public:
   constexpr retargeted_format_context(Context& ctx, OutputIt it);
   retargeted_format_context(retargeted_format_context&&) = delete;
   retargeted_format_context& operator=(retargeted_format_context&&) = delete;
   constexpr ~retargeted_format_context();

   constexpr NewContext& context();
   constexpr void flush();
 };
```

1 *RetargetIt* is an implementation-defined type that models `output_iterator<const typename Context::char_type&>`.

```
constexpr retargeted_format_context(Context& ctx, OutputIt it);
```

2 *Effects*: Initializes *new_context_* such that it holds the same formatting state as `ctx` and such that writing through the iterator yielded by *new_context_*`.out()` will write through `it`, possibly buffered.

```
constexpr ~retargeted_format_context();
```

3 *Effects*: Calls `flush()`.

```
constexpr NewContext& context();
```

4 *Returns*: *new_context_*.

```
constexpr void flush();
```

5 *Effects*: All of the possibly-buffered writes into *new_context_*`.out()` are written through the user-provided output iterator.

§ 4.1.3 An `end_sentry` for `basic_parse_format_context`

In 20.20.6.3 [format.parse.ctx]:

```
namespace std {
  template<class charT>
  class basic_format_parse_context {
```

```
  public:
    using char_type = charT;
    using const_iterator = typename basic_string_view<charT>::const_iterator;
    using iterator = const_iterator;

  private:
    iterator begin_;                                    // exposition only
    iterator end_;                                      // exposition only
    enum indexing { unknown, manual, automatic };       // exposition only
    indexing indexing_;                                 // exposition only
    size_t next_arg_id_;                                // exposition only
    size_t num_args_;                                   // exposition only

  public:
    constexpr explicit basic_format_parse_context(basic_string_view<charT> fmt,
                                                  size_t num_args = 0) noexcept;
    basic_format_parse_context(const basic_format_parse_context&) = delete;
    basic_format_parse_context& operator=(const basic_format_parse_context&) =
        delete;

    constexpr const_iterator begin() const noexcept;
    constexpr const_iterator end() const noexcept;
    constexpr void advance_to(const_iterator it);

    constexpr size_t next_arg_id();
    constexpr void check_arg_id(size_t id);

+   class end_sentry;
  };
}
```

And later:

1 `end_sentry` temporarily reduces the scope of the parse context to facilitate more complex parsing of format specifiers.

2 [*Example*:

```
struct TwoInts {
    int i;
    int j;
};

template <>
struct formatter<TwoInts> {
    formatter<int> fmt_i;
    formatter<int> fmt_j;

    template <class ParseContext>
    constexpr auto parse(ParseContext& ctx) {
        auto it = find(ctx.begin(), ctx.end(), ',');
        if (it == ctx.end()) {
            throw format_error("invalid specifier");
        }

        {
            typename ParseContext::end_sentry _(ctx, it);
            if (fmt_i.parse(ctx) != it) {
                throw format_error("invalid specifier");
```

```
            }
        }

        ++it;
        ctx.advance_to(it);
        return fmt_j.parse(ctx);
    }

    template <class FormatContext>
    auto format(TwoInts ti, FormatContext& ctx) const {
        auto out = ctx.out();
        *out++ = '(';
        ctx.advance_to(out);
        out = fmt_i.format(ti.i, ctx);
        *out++ = ',';
        *out++ = ' ';
        ctx.advance_to(out);
        out = fmt_j.format(ti.j, ctx);
        *out++ = ')';
        return out;
    }
};

print("{:#04x,#06x}\n", TwoInts{222, 173}); // prints (0xde, 0x00ad)
```

-end example]

And

```
template <typename charT>
class basic_format_parse_context<charT>::end_sentry {
    basic_format_parse_context& ctx;   // exposition only
    iterator real_end;                 // exposition only

public:
    constexpr end_sentry(basic_format_parse_context& ctx, iterator it);
    end_sentry(end_sentry&&) = delete;
    end_sentry& operator=(end_sentry&&) = delete;
    constexpr ~end_sentry();
};

constexpr end_sentry(basic_format_parse_context& ctx, iterator it);
```

1 *Effects*: Initializes `ctx` with `ctx` and `real_end` with `ctx.end()`. Assigns `it` to `ctx.end_`.

```
constexpr ~end_sentry();
```

2 *Effects*: Assigns `real_end` to `ctx.end_`

§ 4.1.4 Formatting for ranges

Add to 20.20.1 [format.syn]:

```
namespace std {
  // ...

  // [format.formatter], formatter
  template<class T, class charT = char> struct formatter;
```

```
+ // [format.range], range formatter
+ template<class T, class charT = char>
+   struct range_formatter;
+
+ template<ranges::input_range R, class charT>
+         requires (not same_as<remove_cvref_t<ranges::range_reference_t<R>>, R>)
+           && formattable<ranges::range_reference_t<R>, charT>
+   struct formatter<R, charT>
+     : range_formatter<ranges::range_reference_t<R>, charT>
+   { };

  // ...
}
```

§ 4.1.5 Formatting for `pair` and `tuple`

Add to 20.2.1 [utility.syn]:

```
namespace std {
  // ...

  // [pairs], class template pair
  template<class T1, class T2>
    struct pair;

+ template<class charT, formattable<charT> T1, formattable<charT> T2>
+   struct formatter<pair<T1, T2>, charT>
+     : tuple_formatter<tuple<remove_cvref_t<T1>, remove_cvref_t<T2>>, charT>
+   { };

  // ...
}
```

Add to 20.5.2 [tuple.syn]

```
#include <compare>              // see [compare.syn]

namespace std {
  // [tuple.tuple], class template tuple
  template<class... Types>
    class tuple;

+ template<class charT, formattable<charT>... Types>
+   struct formatter<tuple<Types...>, charT>
+     : tuple_formatter<tuple<remove_cvref_t<Types>...>, charT>
+   { };

  // ...
}
```

Add to 20.20.1 [format.syn]:

```
namespace std {
  // ...

  // [format.formatter], formatter
  template<class T, class charT = char> struct formatter;
```

```
+ // [format.tuple], range formatter
+ template<class Tuple, class charT = char>
+   struct tuple_formatter;

  // ...
}
```

4.1.6 Formatter for `vector<bool>::reference`

Add to 22.3.6 [vector.syn]

```
namespace std {
  // [vector], class template vector
  template<class T, class Allocator = allocator<T>> class vector;

  // ...

  // [vector.bool], class vector<bool>
  template<class Allocator> class vector<bool, Allocator>;

+ template<class R>
+   inline constexpr bool is-vector-bool-reference = see below; // exposition only

+ template<class R, class charT> requires is-vector-bool-reference<R>
+   struct formatter<R, charT>;
```

Add to [vector.bool] at the end:

```
template<class R>
  inline constexpr bool is-vector-bool-reference = see below;
```

8 The variable template *is-vector-bool-reference*`<T>` is `true` if `T` denotes the type `vector<bool, Alloc>::reference` for some type `Alloc` and `vector<bool, Alloc>` is not a program-defined specialization.

```
template<class R, class charT> requires is-vector-bool-reference<R>
  class formatter<R, charT> {
    formatter<bool, charT> fmt;     // exposition only

  public:
    template <class ParseContext>
      constexpr typename ParseContext::iterator
        parse(ParseContext& ctx);

    template <class FormatContext>
      typename FormatContext::iterator
        format(const R& ref, FormatContext& ctx) const;
  };

template <class ParseContext>
  constexpr typename ParseContext::iterator
    parse(ParseContext& ctx);
```

9 *Effects*: Equivalent to `return fmt.parse(ctx);`

```
template <class FormatContext>
  typename FormatContext::iterator
    format(const R& ref, FormatContext& ctx) const;
```

Effects: Equivalent to `return fmt.format(ref, ctx);`

§ 5 References

[fmt-impl] Barry Revzin. 2021. Implementation for range formatting on top of `{fmt}`.
https://godbolt.org/z/fPs1Wxf8E

[LWG3478] Barry Revzin. views::split drops trailing empty range.
https://wg21.link/lwg3478

[LWG3636] Arthur O'Dwyer. formatter::format should be const-qualified.
https://wg21.link/lwg3636

[LWG3651] Barry Revzin. Unspecified lifetime guarantees for the format string.
https://wg21.link/lwg3651

[P1636R2] Lars Gullik Bjønnes. 2019-10-06. Formatters for library types.
https://wg21.link/p1636r2

[P2214R1] Barry Revzin, Conor Hoekstra, Tim Song. 2021-09-14. A Plan for C++23 Ranges.
https://wg21.link/p2214r1

[P2216R3] Victor Zverovich. 2021-02-15. std::format improvements.
https://wg21.link/p2216r3

[P2286R0] Barry Revzin. 2021-01-15. Formatting Ranges.
https://wg21.link/p2286r0

[P2286R1] Barry Revzin. 2021-02-19. Formatting Ranges.
https://wg21.link/p2286r1

[P2286R2] Barry Revzin. 2021. Formatting Ranges.
https://wg21.link/p2286r2

[P2286R3] Barry Revzin. 2021-11-17. Formatting Ranges.
https://wg21.link/p2286r3

[P2290R2] Corentin Jabot. 2021-07-15. Delimited escape sequences.
https://wg21.link/p2290r2

[P2321R2] Tim Song. 2021-06-11. zip.
https://wg21.link/p2321r2

[P2418R0] Victor Zverovich. 2021. Add support for `std::generator`-like types to `std::format`.
https://wg21.link/p2418r0

[P2418R2] Victor Zverovich. 2021-09-24. Add support for std::generator-like types to std::format.
https://wg21.link/p2418r2